

Just-in-Time Vulnerability Detection in Dynamic Web Frameworks via Plain LLMs

Yasin Alperen Bozkurt

September 17, 2026

Abstract

In today’s world, catching security vulnerabilities before they reach production has become critically important for modern technology. Some SAST tools like Brakeman attempt to address this gap by operating within CI/CD pipelines. However, dynamic web frameworks such as Ruby on Rails, Django, and Express.js pose significant challenges for traditional SAST tools. These tools struggle with the dynamic nature of such languages, resulting in high false positive rates and developer fatigue. In this paper, we introduce Erbab, a lightweight JIT vulnerability detection framework that significantly reduces false positive rates by leveraging Plain LLM semantic reasoning. Erbab uses the ‘Plain LLM’ approach with the Gemini 3 Pro model, analyzing code changes (git diff) in Pull Requests to identify and report critical security vulnerabilities. We present a case study within the Ruby ecosystem, demonstrating how this language-agnostic methodology can be effectively implemented. Additionally, we use a pairwise evaluation protocol to evaluate the model’s detection accuracy.

Contents

1	Introduction	3
2	System Design: The ‘Plain LLM’ Architecture	3
2.1	Workflow	3
2.2	Why ‘Plain LLM’?	4
3	Evaluation Plan: A Case Study on Ruby	5
3.1	Dataset Construction: Ruby-JIT-Vul	5
3.2	Pairwise Evaluation Strategy	6
3.3	Preliminary Results (Pilot Study)	6
4	Discussion and Limitations	6
4.1	Inter-File Context Limitations	6
4.2	Validation of the ‘Plain LLM’ Hypothesis	7
4.3	Theoretical Limitations of Execution Monitoring	7
5	Conclusion	7
A	Appendix A: System Prompt	9

1 Introduction

As developers write code, they may inadvertently introduce security vulnerabilities. Early detection of these vulnerabilities in the development lifecycle is therefore essential. The “shift-left” approach addresses this need by incorporating security in the early stages of development [1]. However, in languages like Ruby, Python, and JavaScript, static analysis tools often struggle to interpret these structures accurately when runtime features such as metaprogramming are used. By occasionally generating false positives, these tools create noise and prevent developers from noticing more critical errors.

At this point, Large Language Models (LLMs) come to help us. Unlike SAST tools, LLMs give better results by understanding the purpose of the code. However, current research and benchmarking studies (e.g., JITVul [4]) focus on compiled languages, and this leaves an important gap in evaluating LLMs for modern dynamic web languages.

In this work, we present the `erbab.dev` project, a Just-in-Time vulnerability detection system. `Erbab.dev` adopts a ‘Plain LLM’ strategy that does not require fine-tuning. It performs real-time vulnerability detection by sending code changes (git diff) to the Gemini 3 Pro [11] model through webhooks triggered on Pull Requests. We implement `erbab.dev` on a project using Ruby, an interpreted language, and its web framework Ruby on Rails, to establish a vulnerability detection system. In this paper, we explain how the system works and present our study to test how effective LLMs are in this field.

2 System Design: The ‘Plain LLM’ Architecture

The system design of `Erbab.dev` adopts the “shift-left” and automation-first principles recommended by Thota et al. [1]. Code review is performed automatically while still in the Pull Request stage of development, and detected vulnerabilities are provided as feedback to the developer without any manual intervention. Although our case study is with Ruby, the system works independently of language and framework.

2.1 Workflow

The core workflow operates on a Just-in-Time basis:

1. **Diff Extraction:** Once notified via webhook that a pull request has been created, the system retrieves the git diff information through the API. By focusing only on the changed code rather than the entire repository, this approach reduces analysis costs.
2. **Context-Aware Prompting:** A “Role-Based Prompting” strategy [2] is employed, framing the LLM as a security expert tasked with analyzing the diff for vulnerabilities typical in dynamic frameworks, including SQL Injection, XSS, Mass Assignment, and Insecure Deserialization.
3. **Analysis and Reporting:** The Gemini 3 Pro model [11] processes the diff, posting detected vulnerabilities directly as PR review comments, thereby creating an immediate feedback loop integrated into the developer’s workflow.

Figure 1 shows the complete workflow.

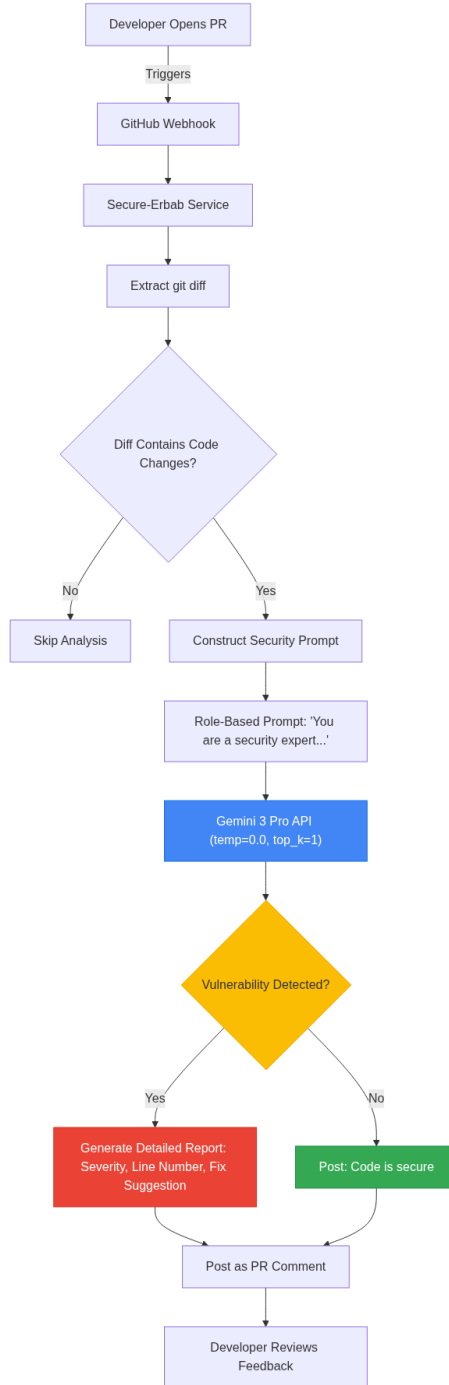


Figure 1: Erbab JIT Vulnerability Detection Workflow

2.2 Why ‘Plain LLM’?

Our approach contrasts with hybrid frameworks such as LSAST [3], which utilize SAST findings to guide LLM analysis. We hypothesize that for dynamic languages like Ruby and Python—where runtime features such as metaprogramming and duck typing render static analysis inherently imprecise—SAST’s high false positive rate may introduce noise into LLM reasoning. In contrast, LLMs can leverage semantic understanding of code intent, potentially reducing false alarms.

Our methodology aligns with Schneider’s characterization of Execution Monitoring

(EM) mechanisms [7], which enforce security policies by observing execution steps and terminating policy-violating executions. Traditional SAST tools, analyzing static code representations, fall outside Schneider’s EM classification. Conversely, Erbab operates on `git diff` outputs—representing actual code changes destined for execution—positioning it conceptually closer to runtime enforcement, albeit applied at the CI/CD stage. This JIT approach enables enforcement of safety properties (e.g., “no SQL injection occurs”) that Schneider proves are EM-enforceable.

To validate this hypothesis, we employ Brakeman [10] (a Ruby SAST tool) as our baseline. While Brakeman relies on control-flow and data-flow graphs for pattern matching, Erbab leverages LLM semantic reasoning to identify context-dependent vulnerabilities that often evade rule-based scanners (see Section 3.3, Case 7). The model is configured with temperature=0.0 and top_k=1 to ensure determinism and reproducibility, and top_p=1.0 for comprehensive sampling.

3 Evaluation Plan: A Case Study on Ruby

To rigorously assess Erbab, we adapt the pairwise evaluation methodology from JITVul [4] to the Ruby ecosystem.

3.1 Dataset Construction: Ruby-JIT-Vul

Given the scarcity of Ruby-focused vulnerability datasets [5], we are constructing **Ruby-JIT-Vul**, comprising 100+ commit pairs from open-source Rails projects such as Discourse, GitLab, and Spree. As an initial step, we have created 7 synthetic test cases to validate our methodology (see Section 3.3).

- **Vulnerable Commit:** Code introducing a vulnerability, e.g.:

```
User.where("name = '#{params[:name]}'")
```

- **Fixed Commit:** The patch resolving the issue

Table 1 illustrates the planned distribution of vulnerability types within the dataset, prioritizing high-impact flaws prevalent in the Ruby on Rails ecosystem (e.g., Mass Assignment). Vulnerability types are classified using the Common Weakness Enumeration (CWE) [9].

Table 1: Planned Distribution of Vulnerability Types in Ruby-JIT-Vul

Vulnerability Type	CWE ID	Count	Description
SQL Injection	CWE-89	25	Unsafe interpolation in ActiveRecord queries
Cross-Site Scripting (XSS)	CWE-79	25	Unescaped user input in ERB templates
Mass Assignment	CWE-915	15	Unpermitted parameters in controller actions
Hardcoded Secrets	CWE-798	15	API keys or tokens committed to git
Insecure Deserialization	CWE-502	10	Unsafe usage of <code>Marshal.load</code> or YAML
Broken Access Control	CWE-284	10	Missing authorization checks (e.g., Pundit)
Total		100	

3.2 Pairwise Evaluation Strategy

We employ a Pairwise Evaluation metric inspired by JITVul [4]. For each vulnerability, the model is presented with two code versions separately: the vulnerable commit (vul-intro) and the fixed commit (vul-fix), without knowledge of which is which. Success is defined as correctly identifying the vulnerable version as unsafe AND the fixed version as safe. This method filters out cases where models flag all code as vulnerable (indicating high false positive rates)—a known limitation of LLM-based vulnerability detection [6]—thereby ensuring high-confidence, actionable insights.

3.3 Preliminary Results (Pilot Study)

To validate the feasibility of our approach, we conducted a pilot study using 7 test cases (6 real vulnerabilities + 1 false positive test) representing common Ruby on Rails vulnerabilities. Gemini 3 Pro (configured with temperature=0.0 and top_k=1) successfully detected 100% of the injected vulnerabilities, demonstrating strong semantic reasoning capabilities.

Table 2: Preliminary Pilot Study Results

Vulnerability Type	CWE ID	Detection Status
SQL Injection (String Interpolation)	CWE-89	Detected
Cross-Site Scripting (XSS)	CWE-79	Detected
Mass Assignment (Permit All)	CWE-915	Detected
Hardcoded Secret (API Key)	CWE-798	Detected
Insecure Deserialization (Marshal)	CWE-502	Detected
Command Injection (<code>Kernel.system</code>)	CWE-78	Detected
Indirect Whitelist (False Positive Test)	CWE-89	Correctly Identified as SAFE

Note on Case 7: The final test case proved particularly insightful. Brakeman flagged it as “SQL Injection (Weak Confidence)” due to its inability to trace data flow through the `extract_sort_parameter()` method. However, Gemini 3 Pro successfully analyzed the entire control flow, recognized the `SAFE_COLUMNS` whitelist validation in the `validate_column()` helper method, and correctly identified the code as safe. This demonstrates the LLM’s superior semantic reasoning capability compared to rule-based SAST tools.

While traditional SAST tools such as Brakeman excel at detecting genuine vulnerabilities, they struggle with context-dependent security measures (e.g., whitelist validation), leading to false positives. A detailed comparative analysis of false positive rates and developer feedback between Erbab and Brakeman is reserved for future work. Our initial findings confirm that a ‘Plain LLM’ approach represents a viable candidate for JIT security auditing.

4 Discussion and Limitations

4.1 Inter-File Context Limitations

Despite promising results, our current ‘Plain LLM’ approach operating solely on `git diff` has inherent inter-file context limitations. A vulnerability might appear in the diff

(e.g., variable usage in a SQL query) while sanitization logic resides in a helper method or different file not included in the PR. This lack of global context can lead to false positives, where safe code is incorrectly flagged as vulnerable due to missing sanitization context.

To mitigate this, future iterations of Erbab will incorporate repository-level context fetching. Using the GitHub API, we will dynamically retrieve definitions of unknown methods or classes referenced in the diff—similar to the “ReAct” agent approach [4]—enabling the LLM to make more accurate assessments without scanning the entire codebase.

4.2 Validation of the ‘Plain LLM’ Hypothesis

Our preliminary study (Case 7) provides initial evidence that LLM semantic reasoning can reduce false positives compared to rule-based SAST. However, a single case is insufficient to conclusively validate our hypothesis that SAST’s high false positive rate is problematic for dynamic languages.

Planned Future Work: We plan to expand Ruby-JIT-Vul to include 20+ ‘SAST False Positive’ cases—code that Brakeman flags as vulnerable but is actually safe due to context-dependent security measures (e.g., whitelists, sanitization in helper methods). This will enable rigorous quantitative comparison of false positive rates between Erbab and Brakeman, providing empirical support for the ‘Plain LLM’ approach in dynamic web frameworks.

4.3 Theoretical Limitations of Execution Monitoring

Schneider [7] formally proves that certain security policies—particularly information flow policies—cannot be enforced by execution monitoring mechanisms, as they require reasoning about alternative possible executions rather than a single observed execution. While Erbab focuses on safety properties (SQL injection, XSS, Mass Assignment, etc.) that are EM-enforceable, it inherits these theoretical limitations. For instance, detecting whether sensitive data from variable x flows to an unauthorized output channel y requires analyzing all possible execution paths, which exceeds the scope of our JIT diff-based approach. Future work could explore hybrid approaches combining static analysis with LLM reasoning to approximate information flow guarantees, though Schneider’s result suggests such policies may require fundamentally different enforcement mechanisms.

5 Conclusion

Erbab demonstrates that a ‘Plain LLM’ approach can effectively address the security challenges of dynamic web frameworks. By leveraging the semantic power of Gemini 3 Pro in a JIT context, we provide a scalable, low-friction alternative to traditional SAST. Our Ruby case study demonstrates the potential for broader application of this methodology across diverse web frameworks.

References

- [1] R. C. Thota, “Cloud-Native DevSecOps: Integrating Security Automation into CI/CD Pipelines,” *Int. J. Innov. Res. Creat. Technol.*, vol. 10, no. 6, pp. 1–17, 2024.
- [2] L. Huynh, Y. Zhang, D. Jayasundera, W. Jeon, H. Kim, T. Bi, and J. B. Hong, “Detecting Code Vulnerabilities using LLMs,” in *Proc. 63rd Annu. Meet. Assoc. Comput. Linguist. (ACL)*, Bangkok, Thailand, July 2025, vol. 1.
- [3] M. Keltek, R. Hu, M. Fani Sani, and Z. Li, “Boosting Cybersecurity Vulnerability Scanning based on LLM-supported Static Application Security Testing,” arXiv:2409.15735, Sep. 2024.
- [4] A. Yildiz, S. G. Teo, Y. Lou, Y. Feng, C. Wang, and D. M. Divakaran, “Benchmarking LLMs and LLM-based Agents in Practical Vulnerability Detection for Code Repositories,” in *Proc. 63rd Annu. Meet. Assoc. Comput. Linguist. (ACL)*, Bangkok, Thailand, July 2025, vol. 1, pp. 30848–30865.
- [5] H. Xu, S. Wang, N. Li, K. Wang, Y. Zhao, K. Chen, T. Yu, Y. Liu, and H. Wang, “Large Language Models for Cyber Security: A Systematic Literature Review,” *ACM Trans. Softw. Eng. Methodol.*, early access, 2025, doi: 10.1145/3769676.
- [6] B. Steenhoek, M. S. Alam, M. M. Rahman, H. Tong, S. Das, M. K. Roy, E. T. Barr, and W. Le, “To Err is Machine: Vulnerability Detection Challenges LLM Reasoning,” arXiv:2403.17218, Mar. 2024.
- [7] F. B. Schneider, “Enforceable security policies,” *ACM Trans. Inf. Syst. Secur.*, vol. 3, no. 1, pp. 30–50, Feb. 2000, doi: 10.1145/353323.353382.
- [8] OWASP Foundation, “OWASP Top Ten,” 2021. [Online]. Available: <https://owasp.org/www-project-top-ten/>
- [9] MITRE Corporation, “Common Weakness Enumeration (CWE).” [Online]. Available: <https://cwe.mitre.org/>
- [10] J. Collins, “Brakeman: A Static Analysis Security Scanner for Ruby on Rails,” 2013. [Online]. Available: <https://brakemanscanner.org/>
- [11] Google DeepMind, “Gemini 3 Pro Model Card,” 2025. [Online]. Available: <https://deepmind.google/models/model-cards/gemini-3-pro>

A Appendix A: System Prompt

The following prompt is used to instruct Gemini 3 Pro for vulnerability detection. It is designed to reduce hallucinations. We do this by limiting analysis to security issues only. We also enforce a structured output format.

You are a security expert. Analyze the following git diff ONLY for SECURITY VULNERABILITIES.

Security Issues to Detect:

1. SQL Injection: Unparameterized queries, SQL built with string interpolation
2. XSS (Cross-Site Scripting): Unsanitized user inputs, raw HTML outputs
3. CSRF: Missing tokens, insecure form handling
4. Hardcoded Secrets: API keys, passwords, tokens, connection strings left in code
5. Injection Vulnerabilities: Command injection, LDAP injection, XML injection
6. Authentication/Authorization: Missing authorization checks, insecure session management
7. Sensitive Data Exposure: Data that shouldn't be logged, unencrypted sensitive information
8. Security Misconfigurations: Debug mode left on, insecure headers
9. Vulnerable Dependencies: Libraries with known vulnerabilities

Output Format:

- Use GitHub markdown format
- List each vulnerability with severity (Critical/High/Medium/Low)
- Specify the line where the issue is found
- Show how to fix it with code example
- If no vulnerability found, write "No security vulnerabilities detected in this change."
- Use details/summary blocks for long explanations
- ONLY report security issues, do NOT write code quality or style suggestions